

PyQUDA

完整技术文档

Python Wrapper for QUDA —GPU-Accelerated Lattice QCD

2026 年 7 月 20 日

项目地址: <https://github.com/CLQCD/PyQUDA.git>

许可证: MIT

Python 版本要求: 3.8+

当前版本: 0.10.53

目录

1	项目概述	3
1.1	设计目标	3
1.2	核心功能	3
1.3	包结构	3
2	安装指南	4
2.1	前置条件	4
2.2	从 PyPI 安装 (推荐)	4
2.3	从源码安装	4
2.4	开发模式安装 (就地构建)	4
3	初始化配置	5
3.1	基本初始化	5
3.2	完整初始化参数	5
3.3	QUDA 环境变量	6
4	格点场类型	6
4.1	LatticeInfo —— 格点几何信息	6
4.2	规范场	7
4.3	费米子场 (Wilson 型)	8
4.4	费米子场 (Staggered 型)	9
4.5	Even-Odd 与 Lexicographic 转换	9
5	Gamma 矩阵代数	9
6	Dirac 算子	10
6.1	类层次结构	10
6.2	GaugeDirac	10
6.3	FermionDirac	11
6.4	具体 Dirac 算子	11
6.5	多网格预条件子	11
7	HMC (混合蒙特卡洛)	12
7.1	作用量类	12
7.2	积分器类	13
7.3	HMC 类	13

8 工具函数 (pyquda_utils)	14
8.1 核心工具 (core.py)	14
8.2 源构造 (source.py)	14
8.3 相位因子 (phase.py / phase_v2.py)	15
8.4 Gamma 工具 (gamma.py)	15
8.5 其他工具	15
9 文件 I/O	16
9.1 低级 I/O API	16
9.2 高级 I/O API (返回 Field 对象)	16
10 插件系统	18
10.1 使用插件构建工具	18
10.2 内置插件	18
10.2.1 pycontract —— 强子收缩	18
10.2.2 pygwu —— GWU 重叠费米子	18
11 示例演示	19
11.1 示例 1: 淬火 HMC	19
11.2 示例 2: 夸克传播子	19
11.3 示例 3: Pion 和质子两点函数	20
11.4 示例 4: Pion PCAC 关系	20
11.5 示例 5: Pion 色散关系	21
12 接口清单 (API Reference)	21
12.1 pyquda 核心包	21
12.2 pyquda.action	22
12.3 pyquda.dirac	22
12.4 pyquda_utils.core	23
12.5 pyquda_utils.source	23
12.6 pyquda_utils.phase / phase_v2	24
12.7 pyquda_utils 其他工具	24
12.8 pyquda_io —— 低级 I/O	25
12.9 pyquda_plugins	26
13 完整示例: 端到端介子谱计算	26
14 依赖关系	27

15 维护与贡献	28
15.1 手动更新注意事项	28
15.2 开发环境建议	28

1 项目概述

PyQUDA 是 QUDA¹ 的 Python 封装，使用 Cython 编写。它使格点 QCD 研究者能够从 Python 中利用 GPU 加速进行高性能计算，同时受益于 CuPy/PyTorch/NumPy 等优化数值库。

1.1 设计目标

- **Python 优先**：所有 API 均为 Pythonic 风格，支持 NumPy/CuPy/PyTorch 数组互操作
- **多 GPU 支持**：通过 mpi4py 实现 MPI 分布式计算
- **多后端**：支持 CUDA、ROCm/HIP、SYCL (Intel GPU) 三种 GPU 后端
- **完整功能**：覆盖夸克传播子、HMC、规范固定、规范 smearing、梯度流等

1.2 核心功能

功能类别	具体内容
文件 I/O	Chroma (QIO/LIME)、MILC、KYU、XQCD、NERSC、OpenQCD、ILDG、NF
夸克传播子	各向同性/异性 Wilson、Clover (含多网格)、HISQ
HMC	规范作用量 + Clover/HISQ 费米子作用量
规范固定	Landau 规范 (over-relaxation 方法)
规范 smearing	3D/4D APE、Stout、HYP
费米子 smearing	Gaussian (Wuppertal)
梯度流	Wilson/Symanzik 流 (含费米子伴随流)
多网格	用于 Wilson/Clover 费米子的几何/代数多网格预条件子
插件系统	自动从 C 头文件生成 Cython 封装

表 1: PyQUDA 核心功能清单

1.3 包结构

PyQUDA 分为三个 PyPI 包：

- **PyQUDA** (`pyquda_core/`)：核心 Cython 绑定，直接封装 QUDA C API

¹QUDA: QCD on CUDA — 基于 CUDA/HIP/SYCL 的格点 QCD GPU 库

- **PyQUDA-Utills** (pyquda_utils/): 高层工具函数 (反演、源构造、相位因子、I/O 辅助)
- **PyQUDA-IO** (pyquda_io/): 低级文件 I/O, 支持 9 种格点数据格式

2 安装指南

2.1 前置条件

组件	要求
Python	≥ 3.8
QUDA	从源码编译安装 (develop 分支, commit $\geq$ PR#1489)
MPI	mpi4py
GPU 后端 (任选一)	CuPy (CUDA/ROCm) 或 PyTorch 或 DPNP (Intel)
编译工具链	Cython, C/C++ 编译器, CUDA/HIP/SYCL 编译器

表 2: 前置条件

2.2 从 PyPI 安装 (推荐)

```
1 export QUDA_PATH=/path/to/quda/build/usqcd
2 python3 -m pip install pyquda pyquda-utils
```

2.3 从源码安装

```
1 export QUDA_PATH=/path/to/quda/build/usqcd
2 git clone --recursive https://github.com/CLQCD/PyQUDA.git
3 cd PyQUDA/pyquda_core
4 python3 -m pip install .
5 cd ..
6 python3 -m pip install .
```

2.4 开发模式安装 (就地构建)

```
1 export QUDA_PATH=/path/to/quda/build/usqcd
2 git clone --recursive https://github.com/CLQCD/PyQUDA.git
3 cd PyQUDA/pyquda_core
4 python3 setup.py build_ext --inplace
```

```

5 cd ..
6 python3 setup.py egg_info
7 ln -s pyquda_core/pyquda_comm ./
8 ln -s pyquda_core/pyquda ./

```

注意：使用开发模式时，需要在运行脚本前将项目根目录添加到 `sys.path`。

3 初始化配置

3.1 基本初始化

PyQUDA 通过 `pyquda.init()` 函数完成所有初始化：

```

1 from pyquda import init
2 from pyquda_utils import core
3
4 # 基本初始化 (4D 格点: 4x4x4x8)
5 init([1, 1, 1, 1], [4, 4, 4, 8], backend="cupy")

```

3.2 完整初始化参数

```

1 def init(
2     grid_size: Optional[List[int]] = None,          # MPI 网格大小
3     latt_size: Optional[List[int]] = None,          # 全局格点大小 [Lx, Ly, Lz, Lt]
4     grid_map: GridMapType = "default",             # MPI 网格映射策略
5     backend: BackendType = "cupy",                 # 数组后端
6     backend_target: str = quda_define.target(),     # GPU 目标
7     init_quda: bool = True,                        # 是否初始化 QUDA
8     use_malloc_quda: bool = False,                 # 使用 QUDA 自定义分配器
9     *,
10    resource_path: str = "",                        # QUDA 资源路径
11    rank_verbosity: List[int] = [0],                # MPI rank 详细度
12    enable_mps: bool = False,                       # 启用 MPS
13    enable_gdr: bool = False,                       # 启用 GPU Direct RDMA
14    enable_p2p: int = 3,                            # P2P 通信策略
15    enable_tuning: bool = True,                     # 启用自动调优
16    enable_device_memory_pool: bool = True,         # 启用设备内存池
17    enable_pinned_memory_pool: bool = True,         # 启用锁页内存池
18    enable_managed_memory: bool = False,            # 启用托管内存
19    deterministic_reduce: bool = False,             # 确定性归约
20    device_reset: bool = False,                     # 设备重置
21    max_multi_rhs: int = 0,                         # 最大多右端量
22    ...
23 )

```

网格映射类型 (grid_map):

- "default" — 默认映射
- "t_fastest" — t 方向最快
- "x_fastest" — x 方向最快
- "minimize" — 最小化通信面
- "shared" — 共享内存
- "dist_graph" — 分布式图

数组后端 (backend): "numpy", "cupy", "torch", "dnp"

精度预设:

- "none" — 双精度所有层级
- "invert" — 混合精度反演
- "multishift" — 多 shift 反演精度
- "multigrid" — 多网格精度配置

3.3 QUDA 环境变量

```
1 export QUDA_RESOURCE_PATH=/path/to/tuning/cache
2 export QUDA_ENABLE_TUNING=1
3 export QUDA_ENABLE_DEVICE_MEMORY_POOL=1
4 export QUDA_ENABLE_PINNED_MEMORY_POOL=1
```

4 格点场类型

PyQUDA 的核心数据结构是格点场 (Lattice Field)，存储在 GPU 显存中，具有特定的 even-odd 排序。

4.1 LatticeInfo —— 格点几何信息

```
1 class LatticeInfo:
2     """
3     描述格点几何的参数容器。
4
5     Nd = 4          # 时空维度
6     Ns = 4          # 自旋分量数
```



```

7     Nc = 3          # 颜色数
8     """
9     def __init__(self, global_size: List[int],
10                  t_boundary: int = 1,
11                  anisotropy: float = 1.0): ...
12
13     # 属性
14     global_size: List[int]      # 全局格点大小 [Lx, Ly, Lz, Lt]
15     size: List[int]            # 本地格点大小
16     grid_size: List[int]       # MPI 网格大小
17     grid_coord: List[int]      # 当前 rank 坐标
18     volume: int                # 本地格点总体积 (Lx*Ly*Lz*Lt // 2)
19     GLt: int                   # 全局时间方向大小
20     Lt: int                     # 本地时间方向大小

```

4.2 规范场

```

1 class LatticeLink(latt_info, value=None):
2     """单方向 SU(3) 链接变量
3     形状: [2, Lt, Lz, Ly, Lx//2, Nc, Nc]"""
4
5 class LatticeGauge(latt_info, L5=4, value=None):
6     """完整规范场 (4 个方向)
7     形状: [4, 2, Lt, Lz, Ly, Lx//2, Nc, Nc]"""
8
9     # 核心方法
10    def use()                # 加载到 QUDA 并返回句柄
11    def pack() / unpack()    # 打包/解包 (用于通信)
12    def covDev(x, mu)        # 协变导数
13    def laplace(x, laplace3D) # 拉普拉斯算子
14
15    # Smearing
16    def apeSmear(n_steps, alpha, ...)
17    def stoutSmear(n_steps, rho, ...)
18    def hypSmear(n_steps, alpha0, alpha1, alpha2, ...)
19
20    # 梯度流
21    def wilsonFlow(flow_time, n_steps, ...)
22    def symanzikFlow(flow_time, n_steps, ...)
23    def wilsonFlowScale(flow_time, n_steps, ...)
24
25    # 可观测量
26    def plaquette()          # 平均 plaquette
27    def polyakovLoop()       # Polyakov 圈

```

```

28     def energy()                # 能量密度
29     def qcharge() / qchargeDensity() # 拓扑荷
30
31     # 规范固定
32     def fixingOVR(n_steps, tol, ...) # Over-relaxation
33     def fixingFFT(n_steps, tol, ...) # FFT 加速
34
35     # 随机数
36     def gauss(seed)              # 高斯随机规范场
37
38     # HDF5 I/O
39     def loadH5(path) / saveH5(path) / appendH5(path) / updateH5(path)
40
41     class LatticeMom(latt_info, L5=4):
42         """动量场 (SU(3) 李代数) """
43
44     class LatticeRotation(latt_info):
45         """规范旋转场 (用于规范固定) """
46
47     class LatticeClover(latt_info):
48         """Clover 项场"""

```

4.3 费米子场 (Wilson 型)

```

1     class LatticeFermion(latt_info):
2         """单个费米子场
3         形状: [2, Lt, Lz, Ly, Lx//2, Ns, Nc]"""
4
5     class HalfLatticeFermion(latt_info):
6         """Even/odd 半费米子场"""
7
8     class MultiLatticeFermion(latt_info, L5):
9         """多右端量费米子场
10        形状: [L5, 2, Lt, Lz, Ly, Lx//2, Ns, Nc]"""
11
12     class MultiHalfLatticeFermion(latt_info, L5):
13         """多右端量半费米子场"""
14
15     class LatticePropagator(latt_info):
16         """完全传播子
17        形状: [2, Lt, Lz, Ly, Lx//2, Ns, Ns, Nc, Nc]"""
18        # 支持 @ 运算符用于 gamma 矩阵乘法

```

4.4 费米子场 (Staggered 型)

```

1 class LatticeStaggeredFermion(latt_info):
2     """Staggered 费米子场, 形状: [2, Lt, Lz, Ly, Lx//2, Nc]"""
3
4 class HalfLatticeStaggeredFermion(latt_info): ...
5 class MultiLatticeStaggeredFermion(latt_info, L5): ...
6 class MultiHalfLatticeStaggeredFermion(latt_info, L5): ...
7
8 class LatticeStaggeredPropagator(latt_info):
9     """Staggered 传播子, 形状: [2, Lt, Lz, Ly, Lx//2, Nc, Nc]"""

```

4.5 Even-Odd 与 Lexicographic 转换

```

1 def lexico(data: NDArray, axes: List[int], dtype=None) -> NDArray:
2     """Even-odd → Lexicographic 排序转换"""
3
4 def evenodd(data: NDArray, axes: List[int], dtype=None) -> NDArray:
5     """Lexicographic → Even-odd 排序转换"""

```

5 Gamma 矩阵代数

```

1 from pyquda import gamma
2
3 class Gamma(index: int, factor: int = 1):
4     """单 gamma 矩阵 (0-15 索引)"""
5     # 运算符: @, +, -, *, /, T, H, D (Dirac 共轭)
6
7 class Projector(factors: list):
8     """16 个 gamma 矩阵的线性组合 (16 元素列表)"""
9     # 支持相同的运算符
10
11 # 预定义常量
12 gamma.gamma_0, gamma.gamma_1, ..., gamma.gamma_4
13
14 # Gamma 基表示
15 class DeGrandRossiMatrix: ... # DeGrand-Rossi (手征) 基
16 class DiracPauliMatrix: ... # Dirac-Pauli 基
17 class DeGrandRossiSparse: ... # 稀疏表示
18 class DiracPauliSparse: ... # 稀疏表示

```

6 Dirac 算子

6.1 类层次结构



6.2 GaugeDirac

```

1 class GaugeDirac(latt_info):
2     """纯规范操作算子"""
3     def loadGauge(gauge)      # 加载规范场
4     def freeGauge()           # 释放规范场
5     def covDev(x, mu)          # 协变导数
6     def laplace(x, laplace3D) # 拉普拉斯算子
7     def wuppertalSmear(x, steps, alpha) # Wuppertal smearing
8
9     # Smearing
10    def apeSmear(n_steps, alpha, ...)
11    def stoutSmear(n_steps, rho, ...)
12    def hypSmear(n_steps, alpha0, alpha1, alpha2, ...)
13
14    # 梯度流
15    def wilsonFlow(flow_time, n_steps, ...)
16    def symanzikFlow(flow_time, n_steps, ...)
17
18    # 可观测量
19    def plaquette() / polyakovLoop() / energy()
20    def qcharge() / qchargeDensity()
21
22    # 规范固定
23    def fixingOVR(n_steps, tol, ...)
24    def fixingFFT(n_steps, tol, ...)
25
26    # 随机数
27    def gaussGauge(seed) / gaussMom(seed)
  
```

6.3 FermionDirac

```

1 class FermionDirac(latt_info, mass, tol, maxiter, multigrid=None):
2     """费米子 Dirac 算子基类"""
3
4     # 基本操作
5     def invert(b) -> x                # 求解  $D * x = b$ 
6     def mat(x) -> b                  # 计算  $D * x$ 
7     def matDagMat(x) -> b            # 计算  $D^\dagger D * x$ 
8     def dslash(x, parity) -> b      # 单 parity Dslash
9
10    # 多右端量版本
11    def invertMultiSrc(b) / matMultiSrc(x) / matDagMatMultiSrc(x)
12    def dslashMultiSrc(x, parity)
13
14    # 预条件版本
15    def invertPC(b)                    # 预条件反演
16    def invertCloverPC(b)              # Clover 预条件反演
17
18    # 配置
19    def useGauge(gauge)                # 上下文管理器加规范场
20    def setPrecision(precision)        # 设置精度
21    def setReconstruct(reconstruct)    # 设置重构方案
22    def setVerbosity(verbosity)        # 设置详细度

```

6.4 具体 Dirac 算子

```

1 class WilsonDirac(latt_info, mass, tol, maxiter, multigrid=None):
2     """Wilson 费米子 Dirac 算子"""
3
4 class CloverWilsonDirac(latt_info, mass, tol, maxiter,
5                         clover_csw=0.0, clover_xi=1.0, multigrid=None):
6     """Clover 改进 Wilson 费米子 Dirac 算子"""
7
8 class StaggeredDirac(latt_info, mass, tol, maxiter,
9                     tadpole_coeff=1.0, multigrid=None):
10    """Staggered 费米子 Dirac 算子"""
11
12 class HISQDirac(latt_info, mass, tol, maxiter,
13                naik_epsilon=0.0, multigrid=None):
14    """HISQ 费米子 Dirac 算子"""

```

6.5 多网格预条件子

```

1 class Multigrid(param, inv_param, instance):
2     """多网格预条件子生命周期管理"""
3     def setParam(coarse_tol, coarse_maxiter,
4                 setup_tol, setup_maxiter,
5                 smoother_nu_pre, smoother_nu_post, smoother_omega)
6     def new() # 初始化
7     def update(thin_update_only) # 更新
8     def destroy() # 销毁

```

7 HMC (混合蒙特卡洛)

7.1 作用量类

```

1 from pyquda.action import (
2     GaugeAction, CloverWilsonAction, HISQAction,
3     MultiHISQAction, LoopParam, RationalParam
4 )
5
6 # 规范作用量
7 gauge_action = GaugeAction(latt_info, loop_param, beta)
8
9 # Clover 费米子作用量 (1 或 2 味)
10 clover_action = CloverWilsonAction(latt_info, rational_param,
11     mass, n_flavor, tol, maxiter, clover_csw)
12
13 # HISQ 费米子作用量
14 hisq_action = HISQAction(latt_info, rational_param,
15     tol, maxiter, naik_epsilon)
16
17 # 多味 HISQ (共享 fat/long 链接)
18 multi_hisq = MultiHISQAction(latt_info, [hisq1, hisq2, ...])

```

LoopParam 和 RationalParam:

```

1 LoopParam(path: List[List[int]], coeff: List[float])
2     """规范圈参数 (如 plaquette [1,0,0,0], rectangle [2,0,0,0])"""
3
4 RationalParam(norm_force, residue_force, offset_force,
5               norm_sample, residue_sample, offset_sample,
6               norm_action, residue_action, offset_action)
7     """有理近似参数 (用于 RHMC) """

```

7.2 积分器类

```

1 class Integrator(n_steps):
2     """积分器基类"""
3
4 class INT_3G1F(n_steps):
5     """3 阶规范 + 1 阶费米子"""
6
7 class 02Nf1Ng0V(n_steps):    # 2 阶, 1 道费米子力 (速度版本)
8 class 02Nf1Ng0P(n_steps):    # 2 阶, 1 道费米子力 (位置版本)
9 class 02Nf2Ng0V(n_steps):    # 2 阶, 2 道费米子力
10 class 02Nf2Ng0P(n_steps):    # 2 阶, 2 道费米子力
11 class 04Nf5Ng0V(n_steps):    # 4 阶, 5 道费米子力
12 class 04Nf5Ng0P(n_steps):    # 4 阶, 5 道费米子力

```

7.3 HMC 类

```

1 class HMC(latt_info, monomials, integrator, hmc_inner=None):
2     """混合蒙特卡洛"""
3
4     def initialize(seed, gauge, mom=None):
5         """初始化轨迹"""
6
7     # 动力学
8     def gaussMom()                # 生成高斯随机动量
9     def integrate(t, project_tol) # 执行轨迹积分
10    def accept(delta_s) -> bool    # Metropolis 接受/拒绝
11
12    # 作用量
13    def momAction()                # 动量作用量
14    def gaugeAction()              # 规范作用量
15    def fermionAction(use_force_param) # 费米子作用量
16
17    # I/O
18    def loadGauge() / saveGauge(path)
19    def loadMom() / saveMom(path)
20
21    # 可观测量
22    def plaquette() / polyakovLoop() / energy()
23
24    # 伪费米子
25    def samplePhi()                # 生成伪费米子
26
27    # HDF5

```

```
28 def loadH5(path) / saveH5(path)
```

8 工具函数 (pyquda_utils)

8.1 核心工具 (core.py)

```
1 # 传播子反演
2 def invert(dirc, source_type, t_srce, source_phase, mrhs, restart) ->
    LatticePropagator
3 def invertEigenvector(dirc, t_srce, source_propag, mrhs, restart)
4 def invertSequential(dirc, source_propag, t_srce, mrhs, restart)
5 def invertPropagator(dirc, source_propag, mrhs, restart)
6
7 # Staggered 版本
8 def invertStaggered(dirc, source_type, t_srce, source_phase, mrhs, restart)
9 def invertStaggeredSequential(...)
10 def invertStaggeredPropagator(...)
11
12 # MPI 数据收集/分发
13 def gatherLattice(data, axes, reduce_op, root)
14 def gatherLattice2(data, tzyx, reduce_op, root)
15 def scatterLattice(data_all, tzyx, root)
16 def gatherScatterLattice(data, tzyx, reduce_op, root)
17
18 # Dirac 算子工厂
19 def getDirac(latt_info, mass, tol, maxiter, xi_0, clover_coeff_t,
20             clover_coeff_r, multigrid) -> FermionDirac
21 def getClover(...) -> CloverWilsonDirac
22 def getWilson(...) -> WilsonDirac
23 def getStaggered(...) -> StaggeredDirac
24 def getHISQ(...) -> HISQDirac
```

8.2 源构造 (source.py)

```
1 def point(latt_info, t_srce, spin, color, phase) -> point 源
2 def wall(latt_info, t_srce, spin, color, phase) -> wall 源
3 def volume(latt_info, spin, color, phase) -> volume 源
4 def momentum(latt_info, t_srce, spin, color, phase) -> 动量源
5 def colorvector(latt_info, t_srce, spin, phase) -> 颜色向量源
6 def source(latt_info, source_type, t_srce, spin, color, source_phase) -> 调度器
7
8 # 便捷函数 (自动生成全自旋/全颜色组合)
```



```

9 def source12(...) # 12 个 spin-color 源 → LatticePropagator
10 def source3(...) # 3 个 color 源 → LatticeStaggeredPropagator
11
12 # Gaussian smearing
13 def gaussianSmear(x, gauge, rho, n_steps)
14 def gaussian(x, gauge, rho, n_steps) / gaussian12(...) / gaussian3(...)
15
16 # Sequential 传播子
17 def sequential(x, t_srce) / sequential12(...) / sequential3(...)

```

8.3 相位因子 (phase.py / phase_v2.py)

```

1 # 动量列表生成
2 def getMomList(mom2_max, mom2_min=0) -> list[tuple]
3 def getMomDict(mom2_max, mom2_min=0) -> dict[int, str]
4
5 # 相位计算
6 class MomentumPhase(latt_info):
7     """GPU 上的动量相位因子"""
8
9 # v2 API
10 class LocationPhase(latt_info): # 格点坐标相位
11 class DistancePhase(latt_info): # 距离相位
12 class MomentumPhase(latt_info):
13     def getPhase(mom, x0) # 单动量相位
14     def getPhases(mom_mode_list, x0) # 多动量批量

```

8.4 Gamma 工具 (gamma.py)

```

1 def gamma_mul_fermion(gamma, fermion) #  $\gamma \times \text{fermion}$ 
2 def propagator_mul_gamma(propagator, gamma) #  $\text{propagator} \times \gamma$ 
3 def gamma_mul_propagator(gamma, propagator) #  $\gamma \times \text{propagator}$ 
4 def gamma_mul_propagator_mul_gamma(g1, prop, g2) #  $\gamma_1 \times \text{propagator} \times \gamma_2$ 
5
6 # Monkey-patch 的运算符支持
7 LatticeFermion @ Gamma # gamma_mul_fermion
8 LatticePropagator @ Gamma # propagator_mul_gamma
9 Gamma @ LatticePropagator # gamma_mul_propagator

```

8.5 其他工具

```

1 # 校验和
2 def checksumSciDAC(field) -> (sum29, sum31)
3
4 # 场类型转换
5 def linkToFermion(link) / fermionToLink(fermion)
6 def multiFermionToPropagator(multi_fermion)
7 def propagatorToMultiFermion(propagator)
8
9 # Wilson 圈
10 def wilson_loop(gauge, path) -> LatticeLink
11
12 # 3D Laplace 本征求解
13 def laplace3d(gauge, n_ev, n_kr, tol, max_restarts, poly_deg, poly_cut) -> (evals,
    evects)
14
15 # FFT (MPI 分布式)
16 def transform(latt_info, field_shape, recvdim, senddim, buf)
17
18 # HMC 参数
19 def wilsonGaugeLoopParam() -> LoopParam
20 def symanzikTreeGaugeLoopParam(u_0) -> LoopParam
21 def symanzikOneLoopGaugeLoopParam(u_0, fermion_type, n_flavor) -> LoopParam
22 def fermionRationalParam(n_flavor, ...) -> RationalParam
23 def staggeredFermionRationalParam(masses, n_flavors, ...) -> RationalParam
24
25 # 多网格验证
26 def real_geo_block_size(sublatt_size, geo_block_size)

```

9 文件 I/O

PyQUDA 支持 9 种格点数据格式,分为低级 API(`pyquda_io`)和高级 API(`pyquda_utils.io`)。

9.1 低级 I/O API

9.2 高级 I/O API (返回 Field 对象)

```

1 from pyquda_utils.io import (
2     # Chroma QIO
3     readChromaQIOGauge, readChromaQIOPropagator, readChromaQIOStaggeredPropagator,
4
5     # ILDG
6     readILDGGauge, readILDGBinGauge,

```

格式	模块	规范场	传播子
Chroma QIO (LIME)	chroma.py	读	读 (Wilson + Staggered)
MILC	milc.py	读/写	读 (Wilson + Staggered)
ILDG (LIME)	ildg.py	读	—
KYU	kyu.py	读/写	读/写
XQCD	xqcd.py	—	读/写
NERSC	nersc.py	读/写	—
OpenQCD	openqcd.py	读/写	—
NPY (NumPy)	np.py	读/写	读/写

表 3: 低级 I/O 格式支持

```

7
8 # MILC
9 readMILCGauge, writeMILCGauge,
10 readMILCQIOPropagator, readMILCQIOStaggeredPropagator,
11
12 # KYU
13 readKYUGauge, writeKYUGauge,
14 readKYUPropagator, writeKYUPropagator,
15
16 # XQCD
17 readXQCDPropagator, writeXQCDPropagator,
18 readXQCDStaggeredPropagator, writeXQCDStaggeredPropagator,
19
20 # NPY
21 readNPYGauge, writeNPYGauge,
22 readNPYPropagator, writeNPYPropagator,
23
24 # NERSC
25 readNERSCGauge, writeNERSCGauge,
26
27 # OpenQCD
28 readOpenQCDGauge, writeOpenQCDGauge,
29
30 # Gamma 基转换
31 rotateToDiracPauli, rotateToDeGrandRossi,
32 )

```

使用示例:

```

1 from pyquda_utils.io import readChromaQIOGauge
2

```

```

3 # 读取 Chroma 格式规范场
4 gauge = readChromaQIOGauge(latt_info, "config.lime", "config.ini.xml")
5
6 # 读取 MILC 格式规范场
7 gauge = readMILCGauge(latt_info, "config.milc")

```

10 插件系统

PyQUDA 的插件系统使用 `pycparser` 自动从 C 头文件生成 Cython 封装。

10.1 使用插件构建工具

```

1 # 自动生成封装
2 pyquda_plugins -l <库名> -i <头文件.h> -L <库路径> -I <包含路径>

```

10.2 内置插件

10.2.1 pycontract —— 强子收缩

```

1 from pyquda_plugins.pycontract import (
2     # 介子
3     mesonTwoPoint, mesonAllSinkTwoPoint, mesonAllSourceTwoPoint,
4
5     # 重子
6     baryonDiquark, baryonTwoPoint, baryonGeneralTwoPoint,
7     baryonSequentialTwoPoint, baryonTwoPoint_v2,
8 )

```

10.2.2 pygwu —— GWU 重叠费米子

```

1 from pyquda_plugins.pygwu import Overlap
2
3 ov = Overlap(latt_info)
4 ov.buildHWilson(gauge, kappa)      # 构造 H_Wilson
5 ov.buildHWilsonEigen(eignum, ...)  # 计算本征向量
6 ov.buildOverlap(prec, use_fp32)     # 构造重叠算子
7 ov.invert(b, masses, tol, maxiter, ...) # 多质量重叠反演

```

11 示例演示

11.1 示例 1：淬火 HMC

```

1 # 1_Quenched_HMC.py
2 from pyquda import init, LatticeInfo, LatticeGauge, LatticeMom
3 from pyquda.action import GaugeAction, LoopParam
4 from pyquda.hmc import HMC, O2Nf1Ng0V
5
6 # 初始化
7 latt_info = LatticeInfo([16, 16, 16, 32])
8 init([1, 1, 1, 1], [16, 16, 16, 32], backend="cupy")
9
10 # 定义作用量 (Wilson 规范作用量)
11 loop_param = LoopParam(path=[[1, 0, 0, 0]], coeff=[4.0 / 3])
12 gauge_action = GaugeAction(latt_info, loop_param, beta=5.8)
13
14 # 定义积分器和 HMC
15 integrator = O2Nf1Ng0V(n_steps=10)
16 hmc = HMC(latt_info, [gauge_action], integrator)
17
18 # 初始化并运行轨迹
19 gauge = LatticeGauge(latt_info)
20 gauge.gauss(seed=42)
21 hmc.initialize(seed=42, gauge=gauge)
22
23 for traj in range(100):
24     hmc.gaussMom()
25     delta_s = hmc.integrate(1.0, 1e-7)
26     if hmc.accept(delta_s):
27         print(f"Traj {traj}: accepted, dS={delta_s:.4f}")
28         hmc.saveGauge(f"config_{traj}.npz")

```

11.2 示例 2：夸克传播子

```

1 # 2_Quark_Propagator.py
2 from pyquda import LatticeInfo, LatticeGauge, LatticeFermion
3 from pyquda.dirac import CloverWilsonDirac
4 from pyquda_utils.io import readChromaQIOGauge
5 from pyquda_utils.core import invert
6 from pyquda_utils.source import wall
7
8 # 初始化格点
9 latt_info = LatticeInfo([24, 24, 24, 72])

```



```

8
9 # 赝标量插入:  $\gamma_5$ 
10 pion_p = mesonTwoPoint(prop, prop, gamma_ab=gamma_4 * gamma_5,
11                        gamma_dc=gamma_4 * gamma_5)
12
13 # PCAC 质量:  $m_q = \langle \partial A_4 \rangle / \langle 2(P) \rangle$ 

```

11.5 示例 5: Pion 色散关系

```

1 # 5_Pion_Dispersion.py
2 from pyquda_utils.phase import MomentumPhase
3
4 phase = MomentumPhase(latt_info)
5
6 for mom_str in ["PX0PY0PZ0", "PX1PY0PZ0", "PX1PY1PZ0", "PX1PY1PZ1"]:
7     mom_phase = phase.getPhase(mom_str, x0=0)
8     # 动量投影的有效质量
9     # meff(t) = arccosh((C(t+1) + C(t-1)) / (2 * C(t)))

```

12 接口清单 (API Reference)

12.1 pyquda 核心包

```

1 # __init__.py —— 初始化
2 init(grid_size, latt_size, grid_map, backend, backend_target, init_quda, ...)
3 initQUDA(grid_size, device, use_malloc_quda)
4 isQUDAInitialized() -> bool
5
6 # field.py —— 格点场类型
7 LatticeInfo(global_size, t_boundary, anisotropy)
8 LatticeInt, MultiLatticeInt
9 LatticeReal, MultiLatticeReal
10 LatticeComplex, MultiLatticeComplex
11 LatticeLink, LatticeGauge, LatticeMom, LatticeRotation, LatticeClover
12 LatticeFermion, HalfLatticeFermion, MultiLatticeFermion, MultiHalfLatticeFermion
13 LatticePropagator
14 LatticeStaggeredFermion, HalfLatticeStaggeredFermion
15 MultiLatticeStaggeredFermion, MultiHalfLatticeStaggeredFermion
16 LatticeStaggeredPropagator
17 lexico(data, axes, dtype), evenodd(data, axes, dtype)
18
19 # gamma.py —— Gamma 代数

```

```

20 Gamma(index, factor)
21 Projector(factors)
22 DeGrandRossiMatrix, DiracPauliMatrix
23 DeGrandRossiSparse, DiracPauliSparse
24
25 # hmc.py —— HMC
26 Integrator(n_steps)
27 INT_3G1F, 02Nf1Ng0V, 02Nf1Ng0P, 02Nf2Ng0V, 02Nf2Ng0P
28 04Nf5Ng0V, 04Nf5Ng0P
29 HMC(latt_info, monomials, integrator, hmc_inner)

```

12.2 pyquda.action

```

1 LoopParam(path, coeff)
2 RationalParam(norm_force, residue_force, offset_force, ...)
3 Action(latt_info, dirac)
4 GaugeAction(latt_info, loop_param, beta)
5 FermionAction(latt_info, dirac, rational_param, mass, n_flavor, tol, maxiter)
6 CloverWilsonAction(latt_info, rational_param, mass, n_flavor, tol, maxiter, clover_csw
   )
7 HISQAction(latt_info, rational_param, tol, maxiter, naik_epsilon)
8 MultiHISQAction(latt_info, hisq_actions)

```

12.3 pyquda.dirac

```

1 Dirac(latt_info)
2 GaugeDirac(latt_info)
3     loadGauge, freeGauge, covDev, laplace, wuppertalSmear
4     apeSmear, stoutSmear, hypSmear
5     wilsonFlow, symanzikFlow, wilsonFlowScale
6     plaquette, polyakovLoop, energy, qcharge, qchargeDensity
7     fixingOVR, fixingFFT, projectSU3, staggeredPhase
8     gaussGauge, gaussMom
9
10 FermionDirac(latt_info, mass, tol, maxiter, multigrid)
11     invert, invertRestart, mat, matDagMat, dslash
12     invertMultiSrc, invertMultiSrcRestart
13     matMultiSrc, matDagMatMultiSrc, dslashMultiSrc
14     invertPC, invertCloverPC, useGauge
15     setPrecision, setReconstruct, setVerbosity
16
17 WilsonDirac(latt_info, mass, tol, maxiter, multigrid)
18 CloverWilsonDirac(latt_info, mass, tol, maxiter, clover_csw, clover_xi, multigrid)

```



```
19 StaggeredDirac(latt_info, mass, tol, maxiter, tadpole_coeff, multigrid)
20 HISQDirac(latt_info, mass, tol, maxiter, naik_epsilon, multigrid)
21
22 Multigrid(param, inv_param, instance)
23     setParam, new, update, destroy
24 Deflation(param, inv_param, instance)
25     new, update, destroy
```

12.4 pyquda_utils.core

```
1 init(...)
2 invert(dirac, source_type, t_srce, source_phase, mrhs, restart)
3 invertEigenvector(dirac, t_srce, source_propag, mrhs, restart)
4 invertSequential(dirac, source_propag, t_srce, mrhs, restart)
5 invertPropagator(dirac, source_propag, mrhs, restart)
6 invertStaggered(dirac, source_type, t_srce, source_phase, mrhs, restart)
7 invertStaggeredSequential(...)
8 invertStaggeredPropagator(...)
9 gatherLattice(data, axes, reduce_op, root)
10 gatherLattice2(data, tzyx, reduce_op, root)
11 scatterLattice(data_all, tzyx, root)
12 gatherScatterLattice(data, tzyx, reduce_op, root)
13 getDirac(latt_info, mass, tol, maxiter, xi_0, clover_coeff_t, clover_coeff_r,
14         multigrid)
15 getClover(...)
16 getWilson(...)
17 getStaggered(...)
18 getHISQ(...)
```

12.5 pyquda_utils.source

```
1 point(latt_info, t_srce, spin, color, phase)
2 wall(latt_info, t_srce, spin, color, phase)
3 volume(latt_info, spin, color, phase)
4 momentum(latt_info, t_srce, spin, color, phase)
5 colorvector(latt_info, t_srce, spin, phase)
6 source(latt_info, source_type, t_srce, spin, color, source_phase)
7 source12(latt_info, source_type, t_srce, source_phase)
8 source3(latt_info, source_type, t_srce, source_phase)
9 gaussianSmear(x, gauge, rho, n_steps)
10 gaussian(x, gauge, rho, n_steps)
11 gaussian12(x12, gauge, rho, n_steps)
12 gaussian3(x3, gauge, rho, n_steps)
```

```
13 sequential(x, t_srce)
14 sequential12(x12, t_srce)
15 sequential3(x3, t_srce)
```

12.6 pyquda_utils.phase / phase_v2

```
1 # phase.py
2 getMomList(mom2_max, mom2_min=0)
3 getMomDict(mom2_max, mom2_min=0)
4 MomentumPhase(latt_info)
5
6 # phase_v2.py
7 LocationPhase(latt_info)
8 DistancePhase(latt_info)
9 MomentumPhase(latt_info): getPhase(mom, x0), getPhases(mom_list, x0)
```

12.7 pyquda_utils 其他工具

```
1 # gamma.py
2 gamma_mul_fermion(gamma, fermion)
3 propagator_mul_gamma(propagator, gamma)
4 gamma_mul_propagator(gamma, propagator)
5 gamma_mul_propagator_mul_gamma(g1, prop, g2)
6
7 # checksum.py
8 checksumSciDAC(field) -> (sum29, sum31)
9
10 # convert.py
11 linkToFermion(link), fermionToLink(fermion)
12 multiFermionToPropagator(multi_fermion)
13 propagatorToMultiFermion(propagator)
14 multiField(fields)
15
16 # wilson_loop.py
17 wilson_loop(gauge, path) -> LatticeLink
18
19 # eigensolve.py
20 laplace3d(gauge, n_ev, n_kr, tol, max_restarts, poly_deg, poly_cut)
21
22 # fft.py
23 transform(latt_info, field_shape, recvdim, senddim, buf)
24 loadBalanceSubsize(GL, size, rank), loadBalanceStart(GL, size, rank)
25
```

```
26 # hmc_param.py
27 wilsonGaugeLoopParam(), symanzikTreeGaugeLoopParam(u_0)
28 symanzikOneLoopGaugeLoopParam(u_0, fermion_type, n_flavor)
29 fermionRationalParam(n_flavor, ...)
30 staggeredFermionRationalParam(masses, n_flavors, ...)
31
32 # geo_block_size.py
33 real_geo_block_size(sublatt_size, geo_block_size)
34
35 # deprecated.py
36 cb2(), smear(), smear4(), invert12(), getDslash(), getStaggeredDslash()
```

12.8 pyquda_io —— 低级 I/O

```
1 # io_general.py
2 IOGeneral, readIOGeneral, writeIOGeneral
3
4 # chroma.py
5 readQIOGauge, readQIOPropagator, readQIOStaggeredPropagator
6
7 # milc.py
8 readGauge, writeGauge, readQIOPropagator, readQIOStaggeredPropagator
9
10 # ildg.py
11 readGauge, readBinGauge
12
13 # kyu.py
14 readGauge, writeGauge, readPropagator, writePropagator
15
16 # xqcd.py
17 readPropagator, writePropagator, readStaggeredPropagator, writeStaggeredPropagator
18 readPropagatorFast, writePropagatorFast
19
20 # nersc.py
21 readGauge, writeGauge
22
23 # openqcd.py
24 readGauge, writeGauge
25
26 # npy.py
27 readGauge, writeGauge, readPropagator, writePropagator
```

12.9 pyquda_plugins

```

1 # pycontract
2 init(), mesonTwoPoint, mesonAllSinkTwoPoint, mesonAllSourceTwoPoint
3 baryonDiquark, baryonTwoPoint, baryonGeneralTwoPoint
4 baryonSequentialTwoPoint, baryonTwoPoint_v2
5
6 # pygwu
7 Overlap(latt_info):
8     buildHWilson, loadHWilsonEigen, buildHWilsonEigen
9     buildOverlap, loadOverlapEigen
10    invert(b, masses, tol, maxiter, ...)
11 readEigenSystem, readEigenSystemInChunks
12 multiFermionFromDiracPauli, multiFermionToDiracPauli, ...

```

13 完整示例：端到端介子谱计算

以下示例整合了 PyQUDA 的核心流程，从读取规范场到提取有效质量：

```

1 from pyquda import init, LatticeInfo, LatticeGauge
2 from pyquda.dirac import CloverWilsonDirac
3 from pyquda_utils.core import invert
4 from pyquda_utils.source import source12
5 from pyquda_utils.io import readChromaQIOGauge
6 from pyquda_plugins.pycontract import mesonTwoPoint
7 from pyquda_utils.phase import MomentumPhase
8 import numpy as np
9
10 # 1. 初始化
11 latt_info = LatticeInfo([24, 24, 24, 72])
12 init([1, 1, 1, 1], [24, 24, 24, 72], backend="cupy")
13
14 # 2. 读取并处理规范场
15 gauge = readChromaQIOGauge(latt_info, "config.lime", "config.ini.xml")
16 gauge.stoutSmear(n_steps=1, rho=0.125)
17
18 # 3. 创建 Dirac 算子
19 dirac = CloverWilsonDirac(latt_info, mass=-0.2395, tol=1e-12,
20                           maxiter=10000, clover_csw=1.0)
21 dirac.setPrecision("invert")
22
23 # 4. 计算传播子
24 prop = source12(latt_info, "wall", t_srce=0,
25                 source_phase=[0, 0, 0, 0])

```

```

26
27 with dirac.useGauge(gauge):
28     prop = invert(dirac, prop, restart=0)
29
30 # 5. 计算 pion 两点函数
31 from pyquda import gamma
32 pion_2pt = mesonTwoPoint(prop, prop,
33     gamma_ab=gamma.gamma_4 * gamma.gamma_5,
34     gamma_dc=gamma.gamma_4 * gamma.gamma_5)
35
36 # 6. 提取有效质量
37 data = pion_2pt.data # numpy/cupy 数组
38 meff = np.arccosh((data[:-2] + data[2:]) / (2 * data[1:-1]))
39 print("Effective mass:", meff)

```

14 依赖关系

依赖	版本	用途
QUDA	develop 分支	GPU 加速格点 QCD 核心库
Python	≥ 3.8	运行环境
mpi4py	—	MPI 分布式计算
numpy	—	数值计算（全后端兼容层）
Cython	—	C/CUDA 绑定生成
可选 GPU 后端（至少一个）		
CuPy	—	CUDA/ROCm GPU 数组
PyTorch	—	CUDA 张量计算
DPNP	—	Intel GPU 数组
可选工具		
h5py	—	HDF5 I/O
opt_einsum	—	张量收缩优化
gmpy2	—	有理近似 (Remez 算法)
matplotlib	—	绑图（示例）
GPT	—	Grid Python Toolkit 互操作

表 4: 完整依赖列表

15 维护与贡献

15.1 手动更新注意事项

以下文件需要手动更新（无法自动生成）：

- `quda.in.pxd` 和 `quda.in.pyx` 中的函数定义
- `pyquda.pyi` 和 `enum_quda.in.py` 中的 docstring

这意味着只要 QUDA API 保持不变，PyQUDA 就能与未来版本兼容。

15.2 开发环境建议

- 使用就地构建模式（`setup.py build_ext --inplace`）而非安装模式
- 修改 Python 文件后立即生效，无需重新编译
- 在集群上运行时需要将项目根目录添加到 `sys.path`